

Programmation fonctionnelle & TDD

Fonctions pures · Immutabilité · Maybe · Either · Railway · TDD · Test doubles · PBT · Tests d'architecture

Deux paradigmes, un seul objectif

OO — Points forts

- Modéliser des entités avec état et cycle de vie (Aggregates, Entities)
- Gérer des règles métier complexes via le polymorphisme
- Modéliser des variations de comportement (Strategy, Visitor)

FP — Points forts

- Transformer des données en pipeline sans duplication
- Tester unitairement sans mocks (fonctions pures)
- Gérer les erreurs explicitement dans les types (Either, Maybe)
- Traitement concurrent naturel (immutabilité)

Pattern hybride

- Couche domaine → OO : Aggregates, Entities, Value Objects
- Transformations → FP : map/ filter/reduce, pipelines
- Gestion erreurs → FP : Either pour les erreurs métier attendues
- Exceptions → pour les erreurs inattendues

Session 5

Programmation fonctionnelle dans un monde objet

Fonctions pures · Immutabilité · HOF · Maybe · Either · Railway

Fonctions pures & Immutabilité

Fonction pure — Définition

- Déterminisme : pour les mêmes arguments, retourne toujours le même résultat
- Absence d'effets de bord : ne modifie aucun état extérieur (pas de BDD, réseau, variables globales)
- Propriété clé : referential transparency — on peut remplacer l'appel par sa valeur
- Conséquence directe : un test est trivial — pas de setup, pas de teardown, pas de mock

Immutabilité — Définition

- Un objet immuable ne peut pas être modifié après sa création
- Toute « modification » produit un nouvel objet avec les nouvelles valeurs
- Absence de bugs liés à l'état partagé — qui a modifié cet objet, et quand ?
- Performance : les GC modernes gèrent efficacement les objets de courte durée de vie

Pourquoi c'est lié aux Value Objects DDD

- Les Value Objects vus hier sont déjà des fonctions pures : `Money.add(other)` retourne un nouveau Money sans modifier l'original
- L'immutabilité est la condition nécessaire pour qu'un Value Object soit auto-validant et thread-safe
- Le lien : DDD impose l'immutabilité dans le domaine ; la PF la généralise à toutes les transformations de données

Higher-order functions · Closures · Currying · Composition

Higher-order functions

- Fonction qui prend d'autres fonctions comme arguments, ou en retourne une
- Exemples : map, filter, reduce
- Permettent d'abstraire des comportements et d'éviter la duplication
- Disponibles nativement en Python, PHP 8, JavaScript, Java 8+

Closures

- Fonction qui capture des variables de son environnement lexical
- Elle transporte avec elle les données dont elle a besoin
- `makeDiscount(0.10)` → retourne une fonction qui applique 10%
- Permet l'application partielle des arguments

Currying & Application partielle

- Currying : transforme `f(a, b)` en `f(a)(b)`
- Application partielle : fixe certains arguments pour créer une nouvelle fonction
- `french_vat = partial(calculate_tax, 0.20)`
- Permet de construire des fonctions spécialisées depuis des fonctions génériques

Composition

- Enchaîne des fonctions : la sortie de l'une devient l'entrée de la suivante
- `pipe(f, g, h)(x) == h(g(f(x)))`
- Equivalent fonctionnel du chaînage de méthodes en OO
- Permet de construire des pipelines lisibles sans variable intermédiaire

Maybe / Option — Valeur potentiellement absente

Définition

Type qui contient soit une valeur (Just / Some), soit rien (Nothing / None). Force le code appelant à gérer l'absence explicitement — impossible d'oublier le `if result is not None`.

Propriétés

- Encapsule une valeur potentiellement absente dans le type de retour
- `map(f)` : applique `f` si la valeur est présente, retourne `Nothing` sinon — jamais d'exception
- `flat_map(f)` : comme `map`, mais `f` retourne elle-même un `Maybe` (chaînage)
- `get_or_else(default)` : extrait la valeur ou retourne un défaut
- Résultat : une chaîne de transformations sans vérification manuelle de `None`

Quand l'utiliser

- Opération de recherche qui peut ne rien trouver : `findById()`, `findByEmail()`
- Configuration optionnelle : `getEnvVar()`, `getConfig(key)`
- Conversion qui peut échouer : `parseInt()`, `parseDate()`
- A distinguer de `Either/Result` : `Maybe` signale l'absence, `Either` signale l'échec avec une raison
- Élimine les `NullPointerException` / `TypeError` silencieux

Either / Result & Railway-Oriented Programming

Either[L, R] — Définition

Contient soit un résultat de succès (Right) soit une valeur d'échec (Left). Contrairement aux exceptions, l'échec est explicite dans le type de retour — le code appelant doit le traiter.

Railway-Oriented Programming (Wlaschin)

Métaphore ferroviaire : chaque étape est un aiguillage — succès = voie principale (Right), échec = voie d'erreur (Left). Les étapes suivantes sont court-circuitées automatiquement.

Pipeline Railway — chaque flat_map() est une étape

validate_quantity

fetch_price

apply_promo

compute_total

Propriétés d'Either

- `map(f)` : applique `f` sur `Right`, propage `Left` inchangé
- `flat_map(f)` : chaîne les étapes susceptibles d'échouer
- `fold(onLeft, onRight)` : traite les deux cas de façon exhaustive

Avantages vs exceptions

- L'échec est visible dans la signature de la fonction
- Le compilateur / type-checker force à traiter le cas d'erreur
- Les erreurs métier attendues ne perturbent pas le flux de contrôle

Session 6

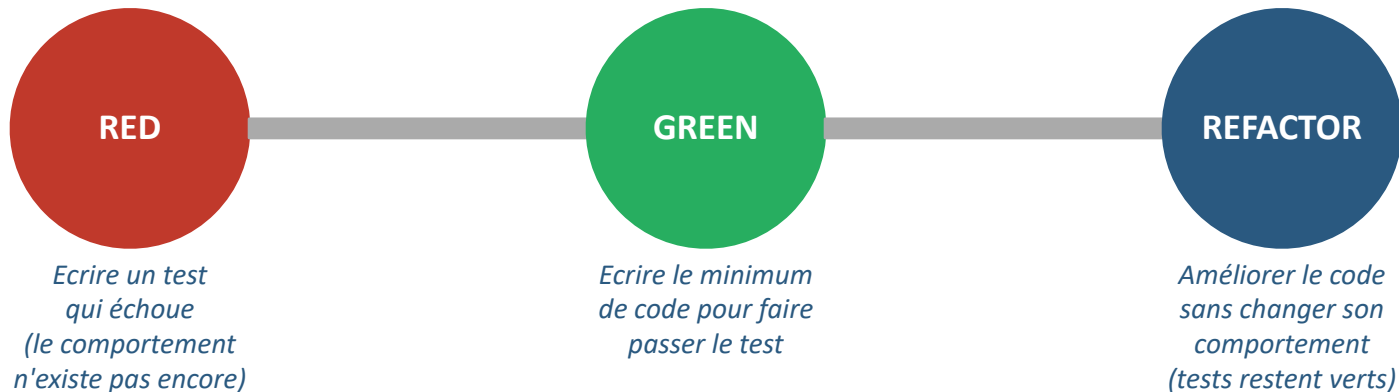
TDD & Tests avancés

Red/Green/Refactor · Test doubles · PBT · Tests d'architecture · Pyramide / Trophy

TDD — Test-Driven Development (Kent Beck, 2002)

Définition

Discipline de développement : écrire un test qui échoue avant d'écrire le code de production. Le cycle Red / Green / Refactor est répété aussi souvent que nécessaire.



Règle fondamentale

- Ne jamais écrire du code de production sans un test qui échoue d'abord — pas d'exception, même pour « un petit fix rapide »

Bénéfices du TDD & Kata TDD

Bénéfices du TDD

- Testabilité par construction : on écrit les tests en premier donc le code est forcément testable
- Documentation vivante : les tests décrivent les comportements attendus mieux que les commentaires
- Refactoring sécurisé : les tests signalent immédiatement toute régression
- Design émergent : écrire le test en premier pousse à concevoir des interfaces avant les détails

Kata TDD — FizzBuzz étendu

- Divisible par 3 → "Fizz"
- Divisible par 5 → "Buzz"
- Divisible par 7 → "Whizz"
- Combinaisons : FizzBuzz, FizzWhizz, BuzzWhizz...
- Objectif : après refactoring, ajouter "Jazz" (×11) sans modifier le code existant (OCP)

Kata recommandés pour s'entraîner

- String Calculator (Roy Oshero) : 8 étapes progressives — idéal pour débiter le cycle Red/Green/Refactor
- Gilded Rose : refactoring d'une fonction de 60 lignes sans casser les tests — le kata legacy le plus célèbre
- Bank Account : Aggregate DDD avec Domain Events — articule TDD et concepts de la veille

Test doubles — Taxonomie de Meszaros (xUnit Test Patterns, 2007)

Type	Définition	Retourne des données	Vérifie les appels	Quand l'utiliser
Dummy	Passé en paramètre mais jamais utilisé	Non	Non	Remplir une signature sans utiliser le paramètre
Stub	Retourne des données préfixées, sans logique	Oui (fixes)	Non	Test a besoin de données en entrée mais ne vérifie pas leur source
Spy	Enregistre les appels pour vérification a posteriori	Optionnel	Oui	Vérifier ce qui a été loggué, émis ou envoyé après exécution
Mock	Vérifie les interactions pendant ou après le test	Optionnel	Oui	Vérifier qu'un comportement A ÉTÉ appelé (n fois, avec ces arguments)
Fake	Implémentation simplifiée mais fonctionnelle	Oui	Non	Plusieurs tests ont besoin d'un comportement cohérent (ex. InMemoryRepository)

Règles de choix des test doubles

Règles de sélection

- Stub : le test a besoin de données mais ne vérifie pas comment elles ont été obtenues
- Mock : le test vérifie qu'une interaction A ÉTÉ produite (sendConfirmation appelé 1 fois)
- Fake : plusieurs tests ont besoin d'un comportement cohérent et persistant entre les appels
- Spy : on inspecte après coup ce qui a été émis (logs, events, emails)
- Dummy : juste remplir une signature — jamais utilisé dans le test

Règle d'or

- Eviter les Mocks sur les classes de votre propre domaine — cela crée un couplage au détail d'implémentation
- Mocker les interfaces / ports est acceptable (Fowler, « Mocks Aren't Stubs », 2007)

Exemple d'application

- InvoiceService.generate() → Stub (données produit prédéfinies)
- PaymentService.process() → Mock (sendConfirmation appelé 1 fois)
- Flux complet sans BDD → Fake (InMemoryOrderRepository)
- AuthService.login() → Spy (WARNING loggué en cas d'échec)

Property-based testing — Au-delà des exemples manuels

Définition

Au lieu de vérifier des exemples choisis manuellement, on décrit des propriétés vraies pour toute entrée valide. Le framework (Hypothesis, QuickCheck, fast-check) génère automatiquement des centaines de cas de test aléatoires.

3 catégories de propriétés

- Comportementales : pour tout multiple de 3, fizzbuzz contient "Fizz"
- Algébriques : commutativité, associativité, élément neutre — `money.add(b) == b.add(money)`
- Round-trip : sérialiser puis désérialiser retourne l'objet original

Shrinking

- Quand un contre-exemple est trouvé, Hypothesis le simplifie automatiquement
- Retourne le plus petit exemple qui reproduit le bug
- Extrêmement utile pour le débogage — on obtient directement le cas minimal qui échoue

Quand l'utiliser

- Value Objects et leurs invariants (Money, Email, DateRange)
- Fonctions pures de calcul
- Parseurs et sérialiseurs (round-trip)
- Algorithmes dont les propriétés mathématiques sont connues

Tests d'architecture — Vérifier le respect des couches

Définition

Tests qui vérifient que les dépendances entre modules respectent les règles architecturales définies. Ils détectent automatiquement les violations avant qu'elles ne s'accumulent.

Exemples de règles vérifiables

- La couche domain n'importe jamais de modules d'infrastructure (SQLAlchemy, PDO, SMTP)
- La couche domain n'importe pas la couche application
- Tout Value Object doit être un dataclass(frozen=True) en Python
- Les classes du namespace Domain ne dépendent pas du namespace Infrastructure
- Les Application Services ne dépendent que d'interfaces, jamais de classes concrètes

Outils

- PHP : ArchUnit-PHP, PHPStan avec règles custom, règle custom via Visitor AST
- Python : inspection avec le module ast de la bibliothèque standard, import-linter
- Java : ArchUnit (référence du genre — 100+ règles prédéfinies)
- Bénéfice : la règle de dépendance de Clean Architecture est vérifiée automatiquement à chaque CI

Pyramide vs Trophy de tests

Pyramide de tests (Mike Cohn, 2009)

- Unitaires (base large) : nombreux, rapides, isolés, peu coûteux
- Intégration (milieu) : testent les interactions entre composants
- E2E (sommet) : peu nombreux, lents, fragiles, coûteux
- Critique : si les unitaires isolent par des mocks, ils testent des détails d'implémentation — fragiles au refactoring

Trophy de tests (Kent C. Dodds, 2018)

- Analyse statique (socle) : linting, typage, tests d'architecture
- Unitaires : pour les fonctions pures et Value Objects — sans mocks
- Intégration (majorité) : testent les comportements réels avec Fakes, sans infrastructure externe
- E2E (quelques-uns) : flux critiques uniquement
- Principe : la confiance, pas la couverture

Stratégie de tests — Recommandation pratique

Ce qu'on teste	Type de test	Double utilisé	Infrastructure ?
Fonctions pures, Value Objects	Unitaire	Aucun	Non
Application Services	Unitaire	Fakes (InMemoryRepository, FakeMailer)	Non
Repository, Adapters	Intégration	BDD de test (SQLite in-memory, Docker)	Oui (légère)
Couches architecturales	Architecture	Analyse statique (ast, ArchUnit, PHPStan)	Non
Flux utilisateur critiques	End-to-end	Aucun	Oui (complète)

Résumé

Concept	Ce qu'il apporte	Lien avec la formation
Fonctions pures	Testabilité triviale, raisonnement local	Complément des Value Objects DDD
Immutabilité	Absence de bugs liés à l'état partagé	Les Value Objects DDD sont déjà immuables
HOF / Composition	Pipelines sans duplication	Alternative au pattern Template Method
Maybe / Option	Absence de valeur explicite dans le type	Elimine les NullPointerException silencieux
Either / Railway	Erreurs métier explicites, pas d'exceptions de flux	Complément à la validation des Aggregates
Cycle TDD	Code testable par construction, design émergent	Complète le refactoring guidé (session 2)
Test doubles	Isolation des unités testées	Permet de tester sans infrastructure
Property-based testing	Exploration exhaustive de l'espace des entrées	Vérifie les invariants des Value Objects
Tests d'architecture	Détection automatique des violations de couches	Renforce la séparation DDD domaine / infra
Pyramide / Trophy	Stratégie globale de couverture	Cadre pour décider quoi tester à chaque niveau